

We have the following domains

- 'const: Any constant value. Can be put anywhere
- 'async: A value with no associated clock, cannot be stored in registers
- T: a named domain. Two named domains cannot be mixed unless there is an explicit $T1 <: T2$ constraint
- (t_1, t_2) : a tuple. Things are a lot easier to reason about if we only have two-tuples so we'll have to do some conversion of n-tuples in the domain inferer
- $t_1 \rightarrow t_2$: Function from one domain to another

This is something we should account for when we do more complex pipelines

- $\text{enabled}(t_1)$: A domain with the same clock as t_1 but with a more restrictive enable.

In addition, we need to have constraints

- !SyncReset
- !AsyncReset
- !Initial
- !Enable

The default domain unless anything else is specified is {}. A unit or language construct can refine this with constraints

Subtyping rules

$$\frac{}{'\text{const} <: t_1} \text{const}$$

$$\frac{}{t_1 <: '\text{async}} \text{async}$$

$$\frac{\Gamma \vdash t_3 : \min(t_1, t_2)}{\Gamma \vdash (t_1, t_2) <: t_3} \text{tuple_sub}$$

$$\frac{\Gamma \vdash t_1 \{c_1 \dots c_n\} \quad \Gamma \vdash t_2 \{c_1, \dots c_n, d_1 \dots\}}{\Gamma \vdash t_1 <: t_2} \text{constraints}$$

Normal rules

$$\frac{}{0 : '\text{const}} \text{literal}$$

$$\frac{}{0 \Leftarrow '\text{const}} \text{literal}$$

$$\frac{t_1 = \text{lookup}(\Gamma, x)}{\Gamma \vdash x : t_1} \text{name}$$

$$\frac{t_1 = \text{lookup}(\Gamma, x)}{\Gamma \vdash x \Rightarrow t} \text{name}$$

$$\frac{\Gamma \vdash e_1 : t_1 \quad \Gamma \vdash e_2 : t_2}{\Gamma \vdash (e_1, e_2) : (t_1, t_2)} \text{tuple_union}$$

$$\frac{\Gamma \vdash e_1 \Rightarrow t_1 \quad \Gamma \vdash e_2 \Rightarrow t_2}{\Gamma \vdash (e_1, e_2) \Leftarrow (t_1, t_2)} \text{tuple_union}$$

$$\frac{\Gamma \vdash e_1 : t_1 \quad \Gamma \vdash \text{has_clock}(t_1)}{\text{reg } x = e_2, \text{extend } (\Gamma, x \in t_1)} \text{clock_requirement}$$

$$\frac{\Gamma e_1 \Rightarrow t_1 \quad \Gamma \vdash x \Leftarrow \text{has_clock}(t_1)}{\text{reg } x = e_1, \text{extend } (\Gamma, x \in t_1)} \text{clock_requirement}$$

$$\frac{\Gamma \vdash e_1 : t_1 \quad \Gamma \vdash e_2 : t_1}{\Gamma \vdash e_1 \oplus e_2 : t_1} \text{binop}$$

$$\frac{\Gamma \vdash e_1 \Rightarrow t_1 \quad \Gamma \vdash e_2 \Leftarrow t_1}{\Gamma \vdash e_1 \oplus e_2 \Rightarrow t_1} \text{binop}$$

$$\frac{\Gamma \vdash e_1 : t_2 \quad \Gamma \vdash e_2 : t_u}{\Gamma \vdash \text{set } e_1 = e_2, t_2 <: t_1} \text{set}$$

The tuple issue

This should domaincheck

```
entity test<'a, 'b>(a: 'a T1, b: 'a T2) {
  reg x = (a, b);
}
```

and this

```
entity test<'a, 'b>(a: 'a T1, b: 'a T2) {
  reg x = (a, 0);
}
```

This should not

```
entity test<'a, 'b>(a: 'a T1, b: 'b T2) {
  reg x = (a, b);
}
```

Case one should be fairly simple, we want to show $x: 'a$

$$\frac{\frac{\frac{}{\text{lookup } (\Gamma, a)}{\text{name}}} \quad \frac{\frac{\frac{}{\text{lookup } (\Gamma, a)}{\text{name}}} \quad \frac{a : 'a \quad b : 'a}{\text{tuple_union}}}{\Gamma \vdash (e_1, e_2) : ('a, 'a)} \quad \text{tuple_sub}}{(a, b) : 'a}$$

Case 2 $(a, 0)$

$$\frac{\frac{\frac{\Gamma \vdash \text{lookup } (\Gamma, a)}{\text{name}}}{\Gamma \vdash a : 'a} \quad \frac{\frac{}{\Gamma \vdash 0 : 'const} \quad \text{const}}{\Gamma \vdash 0 : 'const} \quad \text{tuple_union}}{\Gamma \vdash (a, b) : ('a, 'const)} \quad \text{tuple_sub}}{\Gamma \vdash (a, 0) : 'a}$$

The final path fails for the (a, b) case, because `tuple_sub` requires the subtyping judgement

Set

$a: T, b: 'async; \text{set } a = b$ is not allowed

$$\frac{\frac{}{a : T} \quad \frac{}{b : 'async}}{\Gamma \not\vdash \text{set } a = b, 'async \not\leq T} \text{set}$$

□

Issues

`has_clock` feels sketchy, essentially i'm imagining this:

```

match t {
  'async => false,
  // We could go with false here too, but we can fall back on the anonymous
  // later domain to allow `reg x = x + 1;` without specifying the domain
  'const => true,
  // Named types must have a HasClock constraint which is implicit unless otherwise
  // stated
  T => T.constraints.contains(HasClock),
  // Tuples have a clock if t1 and t2 are in compatible domains. For example
  // (a, 0) can be registered but its domain is (t_1, 'const). Therefore we need
  // a subtyping thing here.
  (t1, t2) => {
    let domain = min(t1, t2); // The smallest subdomain that contains both t1 and t2
    has_clock(domain)
  }
}

```

Just a normal type system with weird subtyping rules

What if we just steal the type system from <https://arxiv.org/pdf/1908.05839> with our subtyping rules and desugaring of our interesting constructs to lambda calculus.

Nope Tuple constructors are weird functions

Operators can be de-sugared to

$$((\lambda o_a : t_1. (\lambda o_b : t_1. o_a))a)b$$

From which we can derive

$$\frac{\frac{\Gamma \vdash \lambda o_a : t_1. (\lambda o_b : t_1. o_a) \Rightarrow t_1 \rightarrow t_1}{\Gamma \vdash (\lambda o_a : t_1. (\lambda o_b : t_1. o_a))a \Rightarrow t_1 \rightarrow t_1} \quad \frac{\Gamma \vdash a \Leftarrow t_1 \rightarrow E \Rightarrow}{\Gamma \vdash b \Leftarrow t_1 \rightarrow E \Rightarrow}}{((\lambda o_a : t_1. (\lambda o_b : t_1. o_a))a)b \Rightarrow t_1}$$

Therefore, we can add a rule that says.

$$\frac{\Gamma \vdash e_1 \Leftarrow t_1 \quad \Gamma \vdash e_2 \Leftarrow t_1}{\Gamma \vdash e_1 \oplus e_2 \Rightarrow t_1}$$

Or wait, can we? We don't have annotations for t_1 . In the least helpful sense, the operator signature is really and then we suddenly need polymorphism 🤔

$$\forall \alpha ((\lambda o_a : \alpha. (\lambda o_b : \alpha. o_a))a)b$$

A much more sane solution is probably to simply change it to,

$$\frac{\Gamma \vdash e_1 \Rightarrow t_1 \quad \Gamma \vdash e_2 \Leftarrow t_1}{\Gamma \vdash e_1 \oplus e_2 \Rightarrow t_1}$$

Can we do the same with set statements? The slightly troubling part is that these do not really have an output type, they constrain the inputs. Is this closer to a variable introduction?

$$((\lambda o_a : t_1. (\lambda o_b : t_1 <: t_2. o_a))a)b$$